

Infrastructure as code*

Distributed Data Processing Environments

Lab Guide 2

This session aims at describing a computing environment in a file, and then at creating and destroying it from that description. It runs entirely on your machine and does not consume credits: in Lab 3 the same description is pointed at the cloud.

Steps First install the tool and describe one container:

1. Install OpenTofu:¹

- On Windows, run: `winget install --exact --id=OpenTofu.Tofu`. If `tofu` is then not found, add `%LOCALAPPDATA%\Microsoft\WinGet\Links` to your `PATH`.
- On Ubuntu, and inside WSL2, run instead:

```
curl -fsSL https://get.opentofu.org/install-opentofu.sh -o install.sh
chmod +x install.sh
./install.sh --install-method deb
```

- Confirm that command `tofu -version` runs.

2. Note that `tofu` runs on your machine, and not inside a container. It talks to the Docker daemon in the same way that it will later talk to a cloud.

3. Create an empty folder next to your Lab 0 folder, and in it a file `main.tf` containing:

```
terraform {
  required_providers {
    docker = { source = "kreuzwerker/docker", version = "~> 4.6" }
  }
}

provider "docker" {}

resource "docker_image" "base" {
  name = "ubuntu:24.04"
}

resource "docker_container" "lab" {
  name      = "adpd-lab1"
  image     = docker_image.base.image_id
  command  = ["sleep", "infinity"]
}
```

4. On Windows, the provider may need to be told where the daemon is. Add `host = "npipe:////./pipe/docker_engine"` inside the provider block.

Now create the environment and observe how it is tracked:

*Predict before prompting throughout. Reviewer mode in the fourth part. See AI protocol.

¹<https://opentofu.org/docs/intro/install/>

1. Run `tofu init`. List the folder contents. What appeared, and why is the provider version recorded in a file?
2. Predict what `tofu plan` will report. Then run it and compare.
3. Run `tofu apply`, then confirm the result with `docker ps`.
4. Run `tofu plan` again and explain why it reports no changes.
5. Stop the container by hand, and run `tofu plan`. Then remove it, and run `plan` again. Explain the difference between the file, the state, and reality.
6. Restore the container, and explain why it is always created after the image, although that order is stated nowhere.
7. Open the state file. What does it record? Ask the agent what such a file may contain in a real deployment, then check its answer against yours.

Now provision the environment and run a program in it:

1. Copy the `Dockerfile` you wrote in Lab 1 into this folder.
2. Change `docker_image` to build it instead of pulling a public image.
3. Mount your Lab 0 folder into the container, so that the dataset and the program are available inside it.
4. Set the container command to run the program, apply, and read its output with `docker logs`.
5. Predict, and then check, what `plan` does when you change the mounted path, and when you change the container name. Which changes replace the container, and which do not?
6. Now change the `Dockerfile` and run `plan` again. Explain why nothing happens, and then use `triggers` to make the image rebuild when its inputs change.

Finally, parameterize the description and have it reviewed:

1. Declare a variable for a parameter of your program, pass it with `-var`, and declare an output for something you want to read back.
2. Ask the agent to criticize your `main.tf`. Rule on each criticism against an actual `plan`.

At the end of the session:

1. Run `tofu destroy`, and confirm with `docker ps -a` that nothing remains.
2. Note that the state file is still there, and that it now records nothing.

Learning Outcomes Apply infrastructure as code to automate creation of computing environments. Distinguish desired, planned, and actual state. Create simple provisioning recipes that install software, download data, and run a program.